First Advanced Topic – Stack-Based Undo System

We chose to research Java stacks to understand how data structures could support an Undo system in our 2048 game. This feature allows players to reverse previous moves without restarting, improving the game's usability. We selected a stack because an Undo operation must reverse moves in the opposite order to which they were made.

Through our research, we learned that stacks follow the Last-In, First-Out (LIFO) principle, where the most recently added element is removed first. In Java, **`push()`** adds an element to a stack, while **`pop()`** removes and returns the most recent element. This makes a stack suitable for Undo because the latest game state must be restored before earlier states. We also learned that storing references to mutable arrays can cause problems, as subsequent modifications may affect previously saved states.

We applied these concepts by implementing an Undo history that stores previous game states, including the board configuration and score. Before a move changes the board, the previous state is captured and only moves that modify the board are added to the Undo history. When Undo is selected, the most recent saved state is retrieved, restoring both the tile positions and their corresponding score. This allows players to reverse multiple moves in the correct order.

A key implementation challenge was ensuring that saved states remain independent of the current board. We addressed this by creating copies of the board rather than storing references to the original array. We also handled empty Undo history so that attempting to undo without previous moves would not cause an error. These decisions helped prevent incorrect state restoration and made the feature more reliable.

Implementing this feature strengthened our understanding of how data structures influence program behavior. We learned that choosing an appropriate structure is only part of the solution; correctly managing mutable data, saving states at the right time, and testing edge cases are equally important. The resulting Undo system improved the game's functionality while allowing us to apply the LIFO principle to a practical problem.

Second Advanced Topic – Git Branching and Collaborative Version Control

Repository: https://github.com/always-pichutan/2048

We chose to research Git branching and collaborative version-control workflows because we were developing 2048 as a team. We wanted to understand how two developers could work independently, combine their contributions, and maintain a stable version of the application.

Through our research, we learned how Git uses commits to record changes and branches to maintain separate lines of development. We also learned the difference between

committing changes locally and pushing them to a remote GitHub repository, where they become available to collaborators. Merging integrates changes from different branches. Together, these concepts support collaborative development while allowing unfinished work to remain separate from the main version.

We applied this workflow using a shared GitHub repository with separate **`Zara`** and **`Selena`** development branches. This allowed us to work on different parts of the application without directly modifying **`main`**. During integration, some separately developed code required debugging. We therefore brought the changes together on the **`Zara`** branch, corrected the issues, and tested the combined application before merging the completed version into **`main`**. Our commit history also provided a record of development and the changes made throughout the project.

An important challenge was that independently developed components did not automatically work correctly when combined. Although branching reduced interference during development, it introduced a need for coordination and integration testing. We learned that merging code is not the same as ensuring that the resulting application functions correctly. Using a development branch for integration allowed us to address problems before incorporating the combined work into **`main`**.

This experience showed us that version control involves more than storing code. We gained practical experience with independent development, change tracking, branch integration, and maintaining a stable main version. We also learned that effective collaboration depends on clear communication, meaningful commits, and testing changes before merging. These practices made our development process more organized and provided a workflow we can apply to future programming projects.